

Web Hosting & Architecture Handbook

From zero knowledge to confidently hosting the EMBRACE AI
dashboard

A complete, plain-English guide — servers, ports, DNS, proxies, IIS, Node.js, databases, CI/CD,
Docker & the Siemens cloud

SIEMENS · Engineering Systems · AI CatalyEST
Edition · July 2026

How to read this handbook

This book assumes you know **nothing** about web servers or hosting. Every term is explained in plain words, with a real-life analogy, before we connect it to our project. You can read cover to cover, or jump to any chapter — each is self-contained.

TIP · Good to know

Boxes like this one highlight useful facts. Amber boxes warn about common mistakes, blue boxes show how a concept maps to OUR project, and purple boxes give a real-life analogy to make an idea click.

WARNING · Watch out

You do NOT need to memorise anything. The goal is understanding. Keep the glossary at the back handy while you read.

Contents

Part 1 — The Foundations

1. What "hosting" really means (and why we need it)
2. What a server is, and what it must provide
3. The client–server model: browser, frontend & backend

Part 2 — The Building Blocks

4. The technology stack, layer by layer
5. Every file in our project, explained
6. The full architecture, end to end

Part 3 — The Network Plumbing

7. localhost, IP addresses & ports
8. DNS: the internet phonebook
9. Routing: how data finds its way
10. Proxies & reverse proxies

Part 4 — Windows Server Hosting

11. IIS: what it is and where it fits
12. ARR & URL Rewrite
13. IIS-based vs non-IIS hosting
14. Node.js, NSSM & Windows services
15. WebSockets & real-time features

Part 5 — Data, Delivery & the Cloud

16. How our database stores & retrieves data
17. CI/CD pipelines & our GitLab redirect
18. Docker, AWS Fargate & the Siemens SDC plan
19. Our two deployment scripts, explained
20. Choosing the right host — decision guide

Reference

- A. Glossary of every term
- B. Quick command runbook

1

What hosting means

and why a laptop is not enough

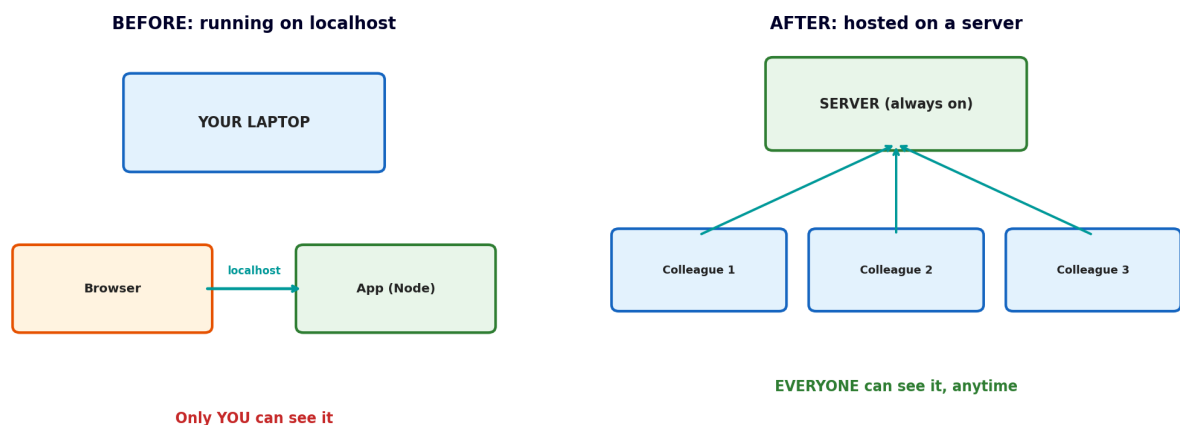
“Hosting” simply means putting your website or app on a computer that is **always switched on and always connected to the network**, so that other people can open it whenever they want — without needing your laptop.

ANALOGY · Real-life analogy

Think of your app like a shop. While you build it on your laptop, the shop is inside your house with the doors locked — only you can walk around it. “Hosting” is like renting a unit on a busy high street: the shop is now on a street everyone can reach, open 24/7, whether or not you are personally there.

Why we cannot just use a laptop

- A laptop gets closed, sleeps, or goes offline — the app would vanish for everyone.
- Home/office networks hide your laptop behind privacy barriers, so colleagues cannot reach it.
- A laptop is not built to safely handle many people connecting at once.



Left: on your laptop only you can see the app. Right: on a server, the whole team can.

NOTE · In our project

We first hosted EMBRACE AI on **Render** (a cloud host) so the whole team could review it at a public link, and we are now planning a permanent, Siemens-internal home. This handbook explains every option.

2

What a server is

the always-on computer behind every website

A **server** is just a computer — but one designed to stay on continuously and to “serve” its files and data to other computers that ask for them. The computers that ask are called **clients** (your browser is a client).

ANALOGY · Real-life analogy

A server is like a restaurant kitchen. Customers (browsers) send orders (requests). The kitchen (server) prepares the dish (data) and sends it back. The kitchen never closes, and it can cook for many tables at once.

What a computer must provide to be a good server

Requirement	Why it matters	Our situation
Always on	If it sleeps, the site goes down for everyone	SN1W7220 is a data-centre server, always on
Stable network + reachable address	Clients must be able to find and reach it	It has a fixed name & IP on the Siemens network
Enough CPU & memory	To handle many visitors smoothly	Server-grade hardware, far beyond a laptop
A web server program	Software that listens for requests	IIS (already there) and/or our Node app
Auto-restart on failure	Recover without a human at 3am	NSSM service + restart rules (Chapter 14)
Security & backups	Protect data and recover from mistakes	Firewall rules + backup scripts in the repo

TIP · Good to know

“Server” can mean the physical machine OR the program running on it (a “web server” like IIS or Node). Same word, two meanings — the context tells you which.

3

The client–server model

browser, frontend and backend

Every web app has two halves that talk to each other:

The frontend (what you see)

The visual part that runs **inside your browser**: the pages, buttons, colours and animations. Built with HTML (structure), CSS (styling) and JavaScript (behaviour).

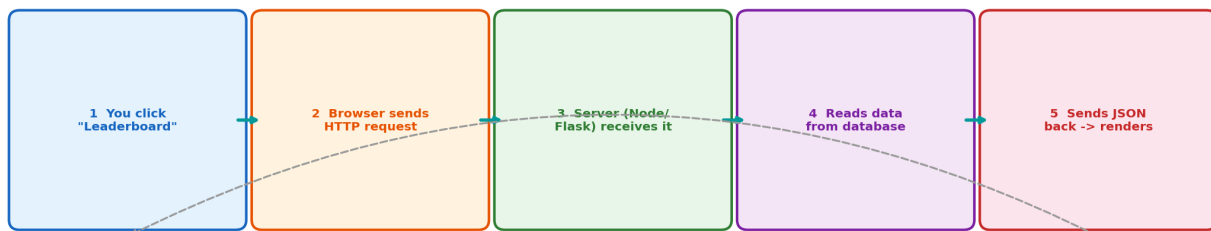
The backend (what you don't see)

The part that runs **on the server**: it holds the real data, applies the rules, and answers the frontend's requests. In our project the backend is written in Node.js (`server.js`) and mirrored in Python/Flask (`app.py`).

ANALOGY · Real-life analogy

The frontend is the dining area you sit in; the backend is the kitchen and store-room. You interact with the dining area, but the food (data) actually comes from the kitchen you never enter.

The journey of ONE request (browser → server → database → back)



The whole round-trip usually takes a few hundredths of a second

A single click becomes a request that travels to the server and comes back as data.

4

The technology stack

every layer and the language it speaks

A “stack” is the set of technologies layered together to make the app work. Here is ours, from what the user sees down to where data rests.

Layer	Technology	Language	What it does
Structure	HTML	HTML	Defines the page content & sections
Styling	CSS	CSS	Colours, layout, glassmorphism, themes
Behaviour	Vanilla JavaScript	JavaScript	Clicks, rendering, quiz logic, side toggle
Real-time	Socket.IO	JavaScript	Live quizzes & surveys (WebSockets)
Backend (main)	Node.js + Express	JavaScript	Serves pages, runs the API
Backend (mirror)	Flask + Flask-SocketIO	Python	Identical API, used on Render
Database	JSON file / MongoDB	JSON	Stores members, points, quizzes, etc.
Process manager	NSSM	—	Keeps the app running as a service
Web server (server)	IIS	—	Optional front door / reverse proxy

NOTE · In our project

We keep **two backends in sync** on purpose: Node (`server.js`) for the Windows server, and Flask (`app.py`) for Render. They expose the exact same API so either can run the app.

5

Every file, explained

what each file is, its extension & tech

A file's **extension** (the letters after the dot) tells the computer what kind of file it is and which program should open it. Here is every important file in the project.

Core application files

File	Ext	Type	Tech	Purpose
<code>server.js</code>	<code>.js</code>	JavaScript source	Node.js	Primary backend: API + WebSockets
<code>app.py</code>	<code>.py</code>	Python source	Flask	Mirror backend (Render)
<code>public/index.html</code>	<code>.html</code>	Web page	HTML	The single-page app shell
<code>public/js/app.js</code>	<code>.js</code>	JavaScript source	Browser JS	All frontend logic
<code>public/css/styles.css</code>	<code>.css</code>	Stylesheet	CSS	All visual styling & themes
<code>data/db.json</code>	<code>.json</code>	Data file	JSON	The flat-file database

Configuration & dependency files

File	Ext	Type	Purpose
<code>package.json</code>	<code>.json</code>	JSON config	Lists Node dependencies & scripts
<code>requirements.txt</code>	<code>.txt</code>	Text list	Lists Python packages to install
<code>Procfile</code>	(none)	Text config	Tells the cloud how to start the app
<code>render.yaml</code>	<code>.yaml</code>	YAML config	Render build/run settings & env vars
<code>.python-version</code>	(dotfile)	Text config	Pins Python 3.12 on Render
<code>.gitlab-ci.yml</code>	<code>.yml</code>	YAML config	The CI/CD pipeline definition
<code>.gitignore</code>	(dotfile)	Text config	Files git should not track
<code>deploy/iis/web.config</code>	<code>.config</code>	XML config	IIS reverse-proxy rules

TIP · Good to know

JSON (JavaScript Object Notation) and **YAML** are both simple, human-readable ways to store settings and data as “key: value” pairs. XML is an older, tag-based format that IIS uses for its `web.config`.

Deployment scripts (the `deploy/windows` folder)

File	Ext	Type	Purpose
<code>host-with-iis.bat</code>	.bat	Batch script	Set up hosting behind IIS (reverse proxy)
<code>host-without-iis.bat</code>	.bat	Batch script	Set up direct Node hosting on a port
<code>setup-server.ps1</code>	.ps1	PowerShell	Install Node + NSSM on the server
<code>install-service.ps1</code>	.ps1	PowerShell	Register the app as a Windows service
<code>update-app.ps1</code>	.ps1	PowerShell	Pull latest code & restart safely
<code>backup-data.ps1</code>	.ps1	PowerShell	Back up db.json & avatars

NOTE · In our project

A **.bat** file is a Windows “batch” script — a list of commands the system runs top to bottom when you double-click it. A **.ps1** file is a more powerful PowerShell script. We use both to make deployment one-click.

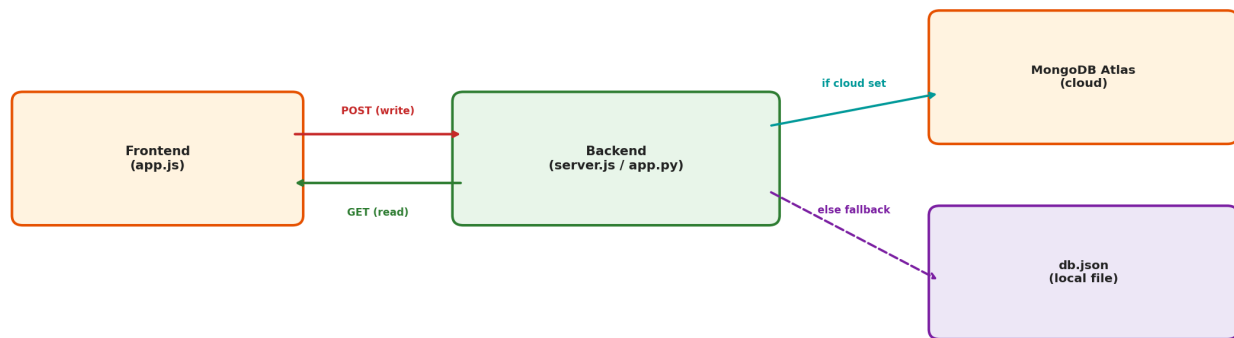
6

The full architecture

how all the pieces connect

Now that you know the parts, here is how they fit together when someone uses the app.

How data flows BOTH ways (read + write) and where it is stored



Data flows in both directions: the frontend reads (GET) and writes (POST) via the backend.

The end-to-end path of a real action

1

You open the site

Your browser requests the page from the server; it returns index.html, styles.css and app.js.

2

The frontend loads data

app.js calls the backend API (for example /api/leaderboard) to fetch the current data.

3

The backend answers

server.js reads the database, computes the result, and returns it as JSON.

4

The page renders

app.js turns the JSON into the visible leaderboard, calendar or quiz on screen.

5

Real-time updates

For live quizzes, a WebSocket keeps an open line so updates appear instantly, with no refresh.

7

localhost, IP & ports

addresses inside and outside a computer

What is localhost?

localhost means “this very computer”. Its special address is `127.0.0.1`. When the app talks to `localhost:3000`, it is talking to a program on the same machine — nobody else on the network is involved.

ANALOGY · Real-life analogy

localhost is like talking to yourself in your own room. Others in the building (the network) cannot hear you unless you open a window (a public port).

What is an IP address?

An **IP address** is a computer's number on a network, like `161.218.195.80`. It is how machines find each other. Names (like `SN1W7220`) are friendlier, and DNS translates the name into the number (Chapter 8).

What is a port?

A single computer runs many network programs at once. A **port** is a numbered door so requests reach the right program. The web uses port **80** (HTTP) and **443** (HTTPS) by default; other programs use other numbers.

ANALOGY · Real-life analogy

The IP address is a building's street address; the port is the specific apartment number inside. “`161.218.195.80:8080`” means “that building, apartment 8080”.

How to choose a port without clashing

- Two programs cannot share the same port — that causes the exact conflict that broke us before.
- Ports 0–1023 are reserved for standard services (80, 443, etc.). Pick a higher one for apps.
- Good app choices: 3000, 8080, 8081. Our scripts default Node to 3000 (private) or 8080 (public).
- Always check a port is free first — our .bat script runs `netstat -ano | findstr LISTENING` to see what is busy.

WARNING · Watch out

On `SN1W7220`, IIS already uses port 80 for Teamcenter and other apps. That is exactly why our scripts put EMBRACE AI on a **separate dedicated port** — so nothing existing is disturbed.

8

DNS: the phonebook

turning names into numbers

Humans remember names; computers use numbers. **DNS** (Domain Name System) is the service that translates a name like `SN1W7220.AD001.SIEMENS.NET` into the IP number the browser actually connects to.

DNS = the internet phonebook (turns a NAME into a NUMBER)



The browser asks DNS for the number behind a name, then connects to that number.

ANALOGY · Real-life analogy

DNS is a phonebook. You know your friend's name (the domain) but to actually call them you need their phone number (the IP). The phonebook does the lookup.

NOTE · In our project

When we tested the server from outside the office, DNS returned a different (VPN) number and the site seemed "down". It was really a name-resolution issue — connecting to the VPN fixed it instantly. DNS problems often masquerade as "the server is down".

9

Routing

how a request finds the right destination

Routing is the general idea of directing a request to the correct place. It happens at two levels in our world:

1. Network routing

Across the network, routers pass your request hop by hop until it reaches the server's IP address — like a parcel moving through sorting centres to a postal address.

2. Application routing

Inside the backend, the code looks at the request's **path** (for example `/api/members` vs `/api/quizzes`) and runs the matching function. Each path is a “route”.

```
app.get('/api/members', ...) // route for the member list
app.post('/api/quizzes', ...) // route to create a quiz
app.get('/api/leaderboard', ...) // route for the leaderboard
```

JAVASCRIPT

ANALOGY · Real-life analogy

Application routing is like a receptionist reading the name on your visitor badge and sending you to the correct department. The path is the badge.

10

Proxies & reverse proxies

the helpful middle-man

A **proxy** is a middle-man that stands between two parties and passes messages between them. A **reverse proxy** sits in front of your app: visitors talk to the proxy, and the proxy quietly forwards them to the real app behind it.

ANALOGY · Real-life analogy

A reverse proxy is like a hotel receptionist. Guests always speak to the front desk (the proxy). The desk relays messages to the right room (your app) and brings the reply back. Guests never need to know the room number.

Why use a reverse proxy at all?

- One front door (port 80/443) can serve many apps behind it, each on its own private port.
- It can add HTTPS, security headers and compression in one place.
- It lets EMBRACE AI live **alongside** Teamcenter on the same IIS server without conflict.

NOTE · In our project

In our IIS option, IIS is the reverse proxy: the browser talks to IIS, and IIS forwards to Node on `127.0.0.1:3000`. This is what the `deploy/iis/web.config` file configures.

11

IIS explained

the web server already on the Windows machine

IIS (Internet Information Services) is Microsoft's built-in web server for Windows. It listens for web requests and decides what to do with them — serve a file, or forward the request somewhere else (a reverse proxy).

Where IIS fits for us

On SN1W7220, IIS is **already running** and other teams depend on it (Teamcenter and more). So IIS must keep running untouched. Our “WITH IIS” option simply adds a new, separate site next to the existing ones, using ARR to forward EMBRACE AI traffic to Node.

WARNING · Watch out

Earlier we tried disabling IIS so Node could take port 80. That is the wrong approach here because it would break Teamcenter. The correct approach is to LET IIS run and put EMBRACE AI beside it — which both our new scripts do.

ANALOGY · Real-life analogy

IIS is like a shopping mall's main entrance and information desk. Many shops (apps) live inside. You do not close the mall to open one new shop — you just add your shop and let the info desk point people to it.

12

ARR & URL Rewrite

the two add-ons that make IIS a proxy

Out of the box IIS serves files. To make it a reverse proxy that forwards to Node, it needs two free Microsoft add-ons:

URL Rewrite

Lets IIS look at an incoming web address and rewrite/redirect it. We use it to say “any request → send to `http://localhost:3000`”.

Application Request Routing (ARR)

Gives IIS the actual ability to forward (proxy) a request to another server and bring the answer back. URL Rewrite decides *where*; ARR does the *forwarding*.

```
<rule name="ReverseProxyToNode" stopProcessing="true">
  <match url="(.*)" />
  <action type="Rewrite" url="http://localhost:3000/{R:1}" />
</rule>
```

[XML](#)

The rewrite rule from our web.config: forward every path to Node on port 3000.

TIP · Good to know

Together, URL Rewrite + ARR turn IIS into a reverse proxy. Both are one-time installs on the server; our `host-with-iis.bat` checks for them and tells you if they are missing.

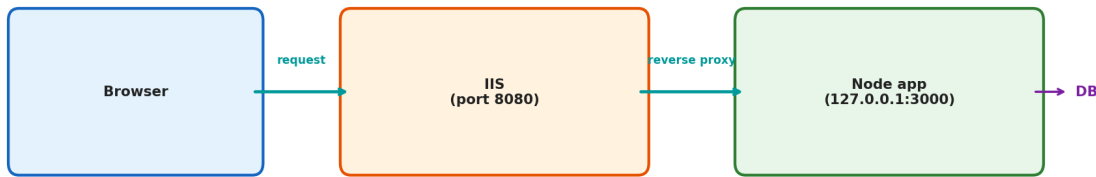
13

IIS vs non-IIS hosting

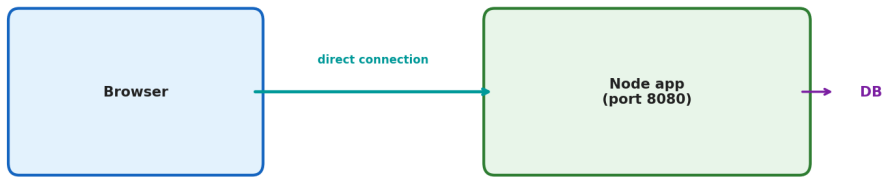
two valid roads to the same destination

We built **two** ready-to-run scripts so your team can pick either approach. Both keep IIS and Teamcenter running; they differ in who answers the browser.

OPTION A — WITH IIS (reverse proxy). IIS keeps serving Teamcenter too.



OPTION B — WITHOUT IIS (direct). Node owns its own dedicated port.



Option A routes through IIS; Option B lets Node answer directly on its own port.

Aspect	WITH IIS (reverse proxy)	WITHOUT IIS (direct Node)
Who answers the browser	IIS, then forwards to Node	Node directly
Port the user sees	IIS site port (e.g. 8080)	Node's port (e.g. 8080)
Node listens on	127.0.0.1:3000 (private)	the public port
Extra setup	ARR + URL Rewrite once	None
HTTPS / headers in one place	Yes (via IIS)	Handled by Node only
Fits Siemens IIS conventions	Yes — most “standard”	Simpler, less conventional
Disturbs Teamcenter?	No	No
Our script	host-with-iis.bat	host-without-iis.bat

TIP · Good to know

If your server team prefers everything behind IIS (common at Siemens), use Option A. If they are happy for the app to use its own port, Option B is simpler. Neither one disables IIS or touches existing sites.

14

Node.js, NSSM & services

running the app reliably, forever

What is Node.js?

Node.js lets JavaScript run **outside** the browser — on a server. It is what executes our `server.js` backend: listening for requests, running the API, and handling WebSockets.

ANALOGY · Real-life analogy

JavaScript used to only work inside browsers. Node.js is like giving that same language a passport to work on servers too — same language, new workplace.

What is a Windows service?

A **service** is a program Windows runs quietly in the background, starting automatically with the machine — no one needs to be logged in. We want our app to be a service so it is always up.

What is NSSM?

NSSM (the “Non-Sucking Service Manager”) is a tiny tool that turns any program — like `node server.js` — into a proper Windows service. It also **restarts the app automatically** if it ever crashes.

Command	What it does
<code>nssm start EmbraceAI</code>	Start the app service
<code>nssm stop EmbraceAI</code>	Stop the app service
<code>nssm restart EmbraceAI</code>	Restart it (after an update)
<code>nssm status EmbraceAI</code>	Check if it is running

NOTE · In our project

Our service is named **EmbraceAI**. We also set Windows’ recovery option so it auto-restarts up to three times on failure — that is what keeps the site up without anyone watching it.

15

WebSockets

the open phone line for real-time features

Normal web requests are one-and-done: ask a question, get an answer, hang up. That is fine for loading a page, but not for a **live quiz** where scores must update instantly for everyone.

A **WebSocket** keeps a **continuous two-way line open** between browser and server, so either side can send updates the moment they happen. We use the **Socket.IO** library on top of WebSockets for our live quizzes and surveys.

ANALOGY · Real-life analogy

A normal request is like sending a letter and waiting for a reply. A WebSocket is like keeping a phone call connected — both people can speak the instant they have something to say.

WARNING · Watch out

Reverse proxies must be told to allow WebSockets. That is why our `web.config` includes `<websocket enabled="true" />` and our IIS script enables the WebSocket feature — otherwise live quizzes would silently fail.

16

How our database works

storing and retrieving data both ways

A **database** is where the app's information lives permanently. Ours can use one of two stores, and the code automatically picks whichever is available:

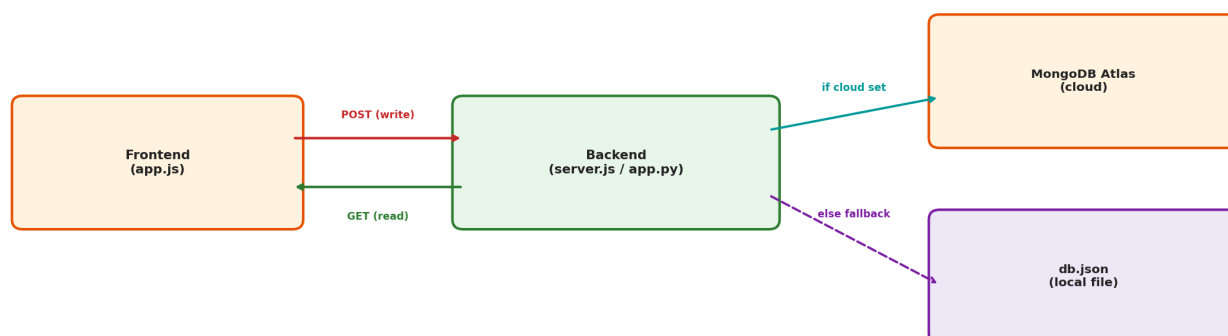
1. db.json — a flat file

A single human-readable JSON file (`data/db.json`) holding all collections: members, events, points, quizzes, surveys and the knowledge board. Perfect for a small internal app and easy to back up (just copy the file).

2. MongoDB Atlas — a cloud database

When an environment variable `MONGODB_URI` is set (as on Render), the same code stores data in MongoDB instead, so data survives redeloys.

How data flows BOTH ways (read + write) and where it is stored



Write with POST, read with GET; data rests in MongoDB if configured, else in db.json.

Reading and writing — both directions

Action	HTTP verb	Example	What happens
Read data	GET	<code>/api/leaderboard?side=engsys</code>	Backend reads the store, returns JSON
Create data	POST	<code>/api/quizzes</code>	Backend adds a record, saves the store
Update data	PUT	<code>/api/events/:id</code>	Backend merges changes into a record
Delete data	DELETE	<code>/api/knowledge/:id</code>	Backend removes a record

NOTE · In our project

Two helper functions do all storage: `readDB()` and `writeDB()` (Node) — mirrored as `read_db()`/`write_db()` in Python. Every route calls these, so switching stores needs no other code changes.

ANALOGY · Real-life analogy

Think of db.json as a well-organised paper ledger in a drawer, and MongoDB as a cloud spreadsheet. The app is trained to write to whichever one is present — the rest of the code never notices the difference.

17

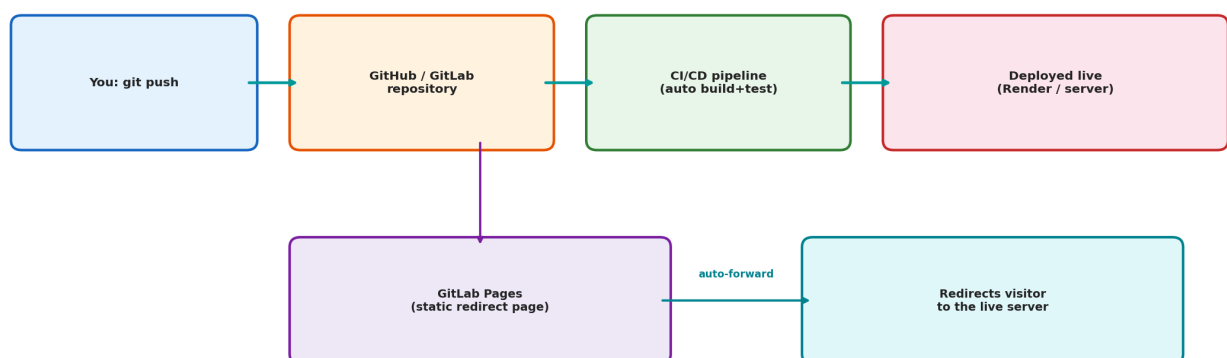
CI/CD & our GitLab redirect

from “git push” to “live”, automatically

What is CI/CD?

CI/CD stands for Continuous Integration / Continuous Delivery. In plain terms: every time you save your code to the shared repository, an automated assistant **checks it and can deploy it** — no manual copying of files.

CI/CD — from “git push” to “live”, plus the GitLab redirect page



You push code; the pipeline validates it; the app deploys; the redirect page points to it.

What is a pipeline?

A **pipeline** is the recipe of automatic steps (defined in `.gitlab-ci.yml`) that run on every push — for example “check the Node code, check the Python code, publish the redirect page.”

How the GitLab URL redirects to our server

GitLab Pages can only host **static** pages, not our live app. So we host a tiny one-page site on GitLab Pages whose only job is to **auto-forward** visitors to the real server. The shareable GitLab link therefore acts as a friendly doorway to the app.

1

You push code

git push sends commits to GitHub (Render) and GitLab (Siemens).

2

The pipeline runs

GitLab validates the code and builds the static redirect page as an artifact.

3

Pages publishes

GitLab serves the redirect page at a stable `code.siemens.io` link.

4

Visitor is forwarded

Opening the link instantly redirects the browser to the live EMBRACE AI server.

NOTE · In our project

We also reduced pipeline “failure” emails by simplifying the checks and only running them on the main branch — noise down, signal up.

18

Docker, Fargate & SDC

the Siemens-cloud action plan

What is Docker?

Docker packages your app together with everything it needs to run (the right Node version, libraries, files) into a single sealed box called a **container**. That container then runs identically on any machine — your laptop, a server, or the cloud.

ANALOGY · Real-life analogy

Docker is like a shipping container. Whatever you pack inside travels intact and fits any ship, train or truck. “It works on my machine” becomes “it works everywhere”, because the machine is packed inside the box.

What is AWS Fargate?

AWS Fargate is a cloud service that runs Docker containers for you without you managing any servers. You hand it your container; it runs and scales it.

What is the Siemens SDC?

The **Siemens Data & AI Cloud (SDC) Project** is an internal, self-service platform that can run containerised apps (via Fargate) and deploy them straight from our GitLab — with Siemens-level security and compliance built in. It is the most promising **internal** permanent home for EMBRACE AI.

What changes if we move to SDC?

Topic	Server approach (SN1W7220)	SDC + Fargate approach
Packaging	Run Node directly via NSSM	Wrap app in a Docker container
Where it runs	On the Windows server	On managed cloud (AWS via SDC)
Scaling	One machine	Scales automatically
Data store	db.json on the server	MongoDB / cloud store (already supported)
Deployment	Run a .bat script	Connect GitLab on the SDC Canvas, click Deploy
IIS involved?	Optionally, as proxy	Not at all

TIP · Good to know

Good news: because our code already supports MongoDB and reads its port from an environment variable, it is **almost** container-ready. Moving to SDC mainly means adding a small Dockerfile — no rewrite.

WARNING · Watch out

SDC provisions real cloud accounts that bill to a cost centre, and needs a Cyber Security Officer and Financial Controller named. Confirm approval and cost with your manager before choosing it as the permanent host.

19

Our two scripts, explained

exactly what each .bat does, step by step

host-with-iis.bat (Option A)

Sets up EMBRACE AI behind IIS as a reverse proxy, without touching any existing site.

1

Checks admin + prerequisites

Confirms it runs as Administrator and that server.js and NSSM exist.

2

Enables IIS WebSocket feature

So live quizzes work through the proxy (safe, idempotent).

3

Verifies ARR + URL Rewrite

Warns you if either add-on is missing.

4

Enables ARR proxy

Turns on server-level reverse-proxying.

5

Points Node to 127.0.0.1:3000

The app listens privately; only IIS reaches it.

6

Creates a dedicated IIS site

A NEW site on its own port — existing sites untouched.

7

Opens firewall + verifies

Allows the public port and tests the API responds.

host-without-iis.bat (Option B)

Runs EMBRACE AI directly on its own dedicated port — IIS keeps running as-is.

1

Checks admin + prerequisites

Same safety checks as Option A.

2

Confirms the port is free

Uses netstat to ensure the chosen port is not busy.

3

Sets Node to the public port

Configures the service's PORT environment variable.

4

Ensures auto-restart

Service auto-starts on boot and restarts on failure.

5

Starts service + opens firewall

Brings the app up and allows the port.

6

Verifies

Tests that the API answers on the chosen port.

NOTE · In our project

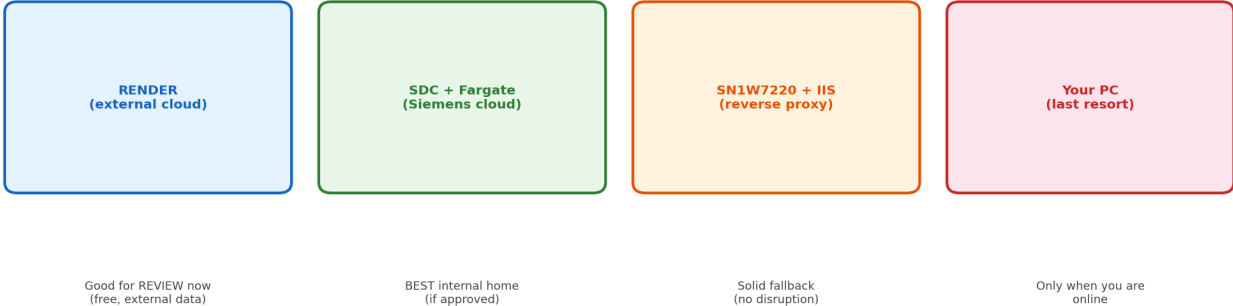
Both scripts are deliberately **non-destructive**: they never stop or disable IIS/W3SVC/WAS, and never modify the Default Web Site or any other site. Teamcenter and colleagues' apps keep running throughout.

20

Choosing the right host

a simple decision guide

The four realistic hosting options, side by side



Four realistic options — each fits a different situation.

If you want...	Best choice	Why
A quick shared link to review now	Render	Already live, auto-deploys, zero effort
A permanent internal Siemens home	SDC + Fargate	Compliant, scalable, deploys from GitLab
To reuse the existing server safely	SN1W7220 + IIS proxy	No new infra, IIS/Teamcenter untouched
A no-frills direct option on the server	SN1W7220 direct port	Simplest; app on its own port
A momentary demo only	Your PC	Fine short-term; only up when you are online

TIP · Good to know

Our recommendation: keep **Render** for review today; pursue **SDC** for the permanent home (pending manager approval); and keep the **IIS reverse-proxy on SN1W7220** ready as a no-disruption fallback — which is exactly why both .bat files now live in the repository.



Glossary

every term, in one place

API (Application Programming Interface)

A set of URLs the frontend calls to get or change data (e.g. `/api/members`).

ARR (Application Request Routing)

An IIS add-on that lets IIS forward requests to another server (reverse proxy).

Backend

The server-side part of the app that holds data and rules (`server.js` / `app.py`).

Batch file (.bat)

A Windows script of commands run top-to-bottom when executed.

CI/CD

Automated checking and deployment of code whenever you push to the repository.

Client

The computer/program making a request — usually your browser.

Container

A sealed box (via Docker) holding the app plus everything it needs to run.

CSS

The language that styles the page — colours, layout, themes.

Database

Where data lives permanently (`db.json` or MongoDB here).

DNS (Domain Name System)

The service that turns a name into an IP address.

Docker

A tool that packages an app into a portable container.

Fargate (AWS)

A cloud service that runs Docker containers without managing servers.

Frontend

The part of the app that runs in the browser (HTML/CSS/JS).

HTML

The language that defines page structure and content.

HTTP / HTTPS

The rules browsers and servers use to talk; HTTPS is the encrypted version.

IIS (Internet Information Services)

Microsoft's built-in web server for Windows.

IP address

A computer's numeric address on a network.

JavaScript

The language for behaviour — in the browser and (via Node) on the server.

JSON

A simple text format for data as key–value pairs.

localhost

A name meaning “this same computer” (address 127.0.0.1).

MongoDB

A cloud database we optionally use instead of db.json.

Node.js

Software that runs JavaScript on a server.

NSSM

A tool that runs a program as an auto-restarting Windows service.

Port

A numbered “door” on a computer so requests reach the right program.

Proxy

A middle-man that relays requests between two parties.

Reverse proxy

A proxy in front of your app that forwards visitor requests to it.

Route

A path (like /api/quizzes) mapped to a function in the backend.

Routing

Directing a request to the right destination (network or code level).

SDC

Siemens Data & AI Cloud — internal platform to host containerised apps.

Server

An always-on computer (or program) that serves data to clients.

Service (Windows)

A background program that starts with the machine.

Socket.IO

A library that provides real-time WebSocket communication.

URL Rewrite

An IIS add-on that rewrites/redirects incoming web addresses.

WebSocket

A persistent two-way connection for instant real-time updates.

YAML / web.config

Configuration formats (YAML for CI/Render; XML web.config for IIS).



Quick runbook

the commands you will actually use

On the server (SN1W7220), as Administrator

```
:: Option A — host behind IIS (reverse proxy)
cd C:\apps\embrace-ai\deploy\windows
host-with-iis.bat

:: Option B — host directly on a dedicated port
host-without-iis.bat
```

BATCH

Update the app to the latest code

```
cd C:\apps\embrace-ai
git pull
npm install
nssm restart EmbraceAI
```

BATCH

Check health

```
nssm status EmbraceAI
curl http://localhost:8080/api/members
netstat -ano | findstr LISTENING
```

BATCH

If something looks down

- Off VPN? DNS may resolve the server name wrong — connect to VPN and retry.
- Port busy? Pick a free port in the .bat file and re-run.
- App crashed? NSSM auto-restarts it; check logs in C:\apps\embrace-ai\logs.
- Seeing an IIS 404? A site/binding is misconfigured — re-run the chosen .bat script.

You made it — you now understand, end to end, how this app is built, hosted and delivered. Keep this handbook by your side; every term you meet in the deployment world is defined in here.